HLD Level 1.8

Reliability & Failure Handling - C++ / Systems Interview Notes

> **Goal: detect partial failure, bound waiting, recover safely, prevent retries from causing duplicate work, and contain overload instead of allowing cascading failure.**

1. Failure is normal

Remote calls can fail through crashes, packet loss, partitions, overload, slow dependencies, connection resets, partial responses, or a lost response after the operation actually succeeded.

```
A -> network -> B -> DB
X X X
```

2. Timeouts

Never wait indefinitely for a remote dependency. Connection, read/write and overall request timeouts bound latency and resource occupancy.

```
A -> B -> wait -> timeout -> recover/fail
```

3. Why missing timeouts hurt

Stuck calls hold threads, sockets, buffers and connection-pool entries. Enough blocked work can exhaust the caller and create a cascade.

```
Slow DB -> blocked threads -> queue growth -> API failure
```

4. Timeout selection

Too short causes false failures/retries; too long ties up resources. Choose using latency expectations, SLOs and the caller's remaining deadline.

5. Deadline propagation

An overall request deadline should constrain downstream calls; each service should not independently wait longer than the original budget.

```
Client budget 2s -> A uses 300ms -> ~1.7s remains
```

6. Retries

Retries help transient failures, but not permanent failures or overload. Retry only when the operation is safe and time remains.

```
request -> transient fail -> retry -> success
```

7. Retry storms

Many callers retrying an unhealthy dependency multiply traffic exactly when capacity is lowest.

```
10k requests x repeated retries -> larger outage
```

8. Exponential backoff

Increase delay between attempts rather than retrying immediately.

```
100ms -> 200ms -> 400ms -> 800ms
```

9. Jitter

Randomize retry timing so thousands of clients do not retry simultaneously.

```
same schedule -> spike; jitter -> spread
```

10. Bounded retries

Limit attempts or use the overall deadline. After exhaustion, fail, queue for later, DLQ, or degrade according to semantics.

11. Idempotency

A timeout means the result may be unknown: the server may have completed the operation but the response was lost. Retrying state-changing work requires idempotency/deduplication.

```
CreateOrder succeeds -> response lost -> retry must not create second order
```

12. Circuit breaker

Stop calling a dependency after repeated failures; fail fast while open, then allow limited probes in half-open state.

```
CLOSED -> failures -> OPEN -> cooldown -> HALF-OPEN -> CLOSED
```

13. Bulkheads / isolation

Separate pools/queues/limits can prevent one failing dependency from consuming all process resources.

```
Pool A -> dependency A
Pool B -> dependency B
```

14. Health checks

Liveness asks whether the process is alive; readiness asks whether it can currently serve traffic. Alive but DB-unavailable can mean not ready.

15. Heartbeats

Periodic evidence of liveness/connectivity. Common in device fleets and persistent connections. Associate heartbeat with the correct current process/connection identity.

```
Device -> heartbeat -> Hub; misses beyond threshold -> suspect/offline
```

16. Heartbeat tuning

Frequent heartbeats detect faster but cost more and may false-positive; long intervals delay detection. Usually tolerate multiple misses.

17. Failover

When an active component fails, work can move to another healthy instance. Define detection, takeover, state and client redirection.

```
Primary A fails -> B promoted / traffic redirected
```

18. Graceful degradation

Preserve core functionality when a noncritical dependency fails. Example: serve product data without recommendations.

```
Core data OK + recommendations down -> partial useful response
```

19. Fallbacks

Possible fallbacks include cached/stale data, defaults, alternate replicas/providers or asynchronous completion, but only when semantically safe.

20. Load shedding

Under overload, deliberately reject lower-priority/excess work rather than let all requests become extremely slow and exhaust resources.

```
capacity 20k/s, incoming 35k/s -> reject excess quickly
```

21. Rate limiting

Control how much traffic a client/system may send. Conceptual algorithms include token bucket, leaky bucket and fixed/sliding windows.

22. Backpressure

Constrain upstream when downstream cannot keep up using bounded queues, blocking, rejection, throttling or flow control.

```
producer 50k/s > consumer 40k/s -> must constrain or backlog grows
```

23. Cascading failures

A slow dependency blocks callers; queues grow; callers retry; more components overload. Timeouts, bounds, circuit breakers and shedding contain cascades.

```
DB slow -> API blocks -> retries -> wider outage
```

24. Graceful shutdown

Mark not ready, stop accepting new work, drain in-flight work, stop workers safely, close resources, then exit.

```
NOT READY -> stop new work -> drain -> close -> exit
```

25. Crash recovery

After a crash, reason about durable state, in-flight work, replay/duplicates, temporary resources and safe startup recovery.

26. OTA reliability

A/B update design preserves a known-good bank while installing and validating the new version, then rolls back if health checks fail.

```
validate -> inactive bank -> verify -> switch -> health check -> rollback if needed
```

27. Combined policy

Timeouts, bounded retries with backoff/jitter, circuit breakers and fallbacks solve different failure modes and work together.

```
deadline -> call -> retry transient -> repeated failure -> circuit open -> fallback/fail fast
```

28. Observability

Track timeout rate, retry count, circuit state, queue depth, dependency latency, heartbeat misses, reconnect rate and failover events.

29. C++ resource exhaustion

Pending work consumes concrete resources: heap buffers, threads, FDs, sockets and pool slots. Bounded queues/timeouts/cancellation are memory/resource controls too.

```
100k pending x 32KB buffer ~= 3.2GB payload buffers
```

30. Reconnect storms

After a gateway failure/restart, thousands of devices can reconnect together. Exponential backoff plus jitter spreads authentication/TLS/connection setup load.

31. Failure domains

Redundancy only helps if copies fail independently. Two replicas sharing the same machine, power or network path may share one failure domain.

32. Interview checklist

1. What can fail?
2. How is it detected?
3. Timeout/deadline?
4. Is retry safe?
5. Retry count?
6. Backoff + jitter?
7. Idempotency?
8. Circuit breaker?
9. Health/readiness?
10. Heartbeat threshold?
11. Failover?
12. Fallback/degradation?
13. Bounded resources?
14. Load shedding/backpressure?
15. Graceful shutdown/recovery?
16. What metrics expose failure?

33. Common mistakes

No timeout: callers can hang/exhaust resources.
Retry everything: overload/permanent errors worsen.
Immediate retry: creates storms.
Timeout means server did nothing: false; result may be unknown.
Blind retry of non-idempotent write: duplicate effects.
Heartbeat is perfect failure detector: false.
Fallback at any cost: stale/incorrect data may be unsafe.
Unbounded queue: overload becomes memory/latency failure.

34. Practice scenarios

1. A payment times out after the service may already have charged the card. Should you simply retry? What do you need?

2. 10,000 devices lose connection and retry exactly every second. What problem occurs and how do you improve it?

3. Recommendation service is down but product DB is healthy. Must the product page fail?

4. A downstream API has failed for five minutes. Why can a circuit breaker help?

5. Producer sends 50k/s while consumer permanently handles 40k/s. Is retry the solution? What mechanisms matter?

6. Process is alive but DB pool is exhausted. Liveness or readiness problem?

7. Explain your A/B OTA rollback as a reliability design.

35. 60-second revision

Timeout: bound waiting/resource occupancy.
Retry: recover transient failure when safe.
Backoff: increasing delay.
Jitter: spread synchronized retries.

Idempotency: repeat does not duplicate effect.
Circuit breaker: stop hammering unhealthy dependency.
Heartbeat: periodic liveness evidence.
Readiness: currently able to serve.
Failover: move work to healthy replacement.
Graceful degradation: preserve core functionality.
Load shedding: reject excess deliberately.
Backpressure: constrain upstream.
Graceful shutdown: stop new work, drain, release safely.

36. Ready for Level 1.9 when...

You can design timeout/retry policies, explain idempotency, use backoff/jitter/circuit breakers appropriately, reason about heartbeat/readiness/failover, and contain overload with bounded resources, degradation and backpressure.